Harness

AI 概念一直在演進，從早期的 Prompt Engineering 到現在的 "Harness Engineering"，這背後反映了我們對 AI 理解的不斷加深。

前景提要：如果你已经听说过 AI Agent（智能体），你可能会觉得 Agent 的核心就是大模型本身。但实际上，真正让 Agent 能在现实世界中干活的，不是大模型的"脑子"，而是包裹在它外面的那一整套"骨架和装备"。这套东西，就叫 Harness（运行框架）。

今天的内容，我们的目标是搞懂三个问题：Harness 是什么？它为什么会出现？它里面都有什么？

1. 什么是 AI Agent?

AI Agent = 大语言模型（LLM）+ 外部装备增强

核心作用：让大模型具备**记忆信息、调用工具、分步执行**的能力，是 LLM 的进阶形态。

具体的内容可以结合我前两天的笔记去学习。

2. Harness 的定义与作用

Harness 是 AI Agent 的 **软件基础设施**，统管大模型之外的全流程事务。

核心职责：

- 信息输入输出解析
- 工具执行调度
- 记忆管理
- 安全管控
  形象类比：如同马具、安全带，**连接、约束、协同**Agent 所有组件，保障稳定运行。

3. Harness 的起源

出现背景：AI 从**简单问答**转向 **复杂自主任务**，需要标准化支撑能力。

大模型原有的痛点：1.模型没有手脚 2.模型会失忆 3.模型会跑偏

针对于LLM的这些痛点内容，Harness给出了它的解决方案：

1. 初始化 Agent：第一次运行时，设置好项目结构、功能列表、进度文件
2. 增量推进：每次只做一个小功能，做完提交代码、更新进度
3. 验证机制：每次开始新会话时，先检查之前的工作是否正常
4. 防止过早结束：维护一个功能清单，只有全部通过测试才算完成

演进路径：ChatGPT 插件系统 → Anthropic/OpenAI 相关研究 → Harness 概念成熟落地。

核心价值：补齐工具调用、上下文管理、流程控制三大关键能力。

4. Harness 核心组件

1. **Agent 循环**
   信息组装→模型思考→动作执行→结果观察，闭环迭代直至完成任务。
2. **工具系统**
   提供真实世界操作能力，支持文件处理、命令执行等工具调用。
3. **上下文管理**
   维护对话历史、信息压缩过滤，保留关键有效信息。

4. **安全系统**
   多层防护机制，禁止危险操作，保障执行安全。
5. **会话与持久化**
   存储对话记录与任务进度，支持跨会话持续记忆。

5. Scaffolding 与 Harness 的区别

- **Scaffolding（脚手架）**：Agent 运行前阶段，负责系统提示词编译、工具注册等组装工作。
- **Harness（运行框架）**：Agent 运行时阶段，负责工具调度、上下文管理、安全校验、状态保存全流程协调。

6. Harness 核心价值

Harness 是 AI Agent 完成复杂任务的 **关键支撑**，通过标准化组件体系，将单一大模型转变为可**长期自主运行、稳定执行**的智能体。

7.对我们的启发

即使你不打算从零构建一个 Harness，理解这些概念也有很大价值：

1. 使用 Cursor、Claude Code 等工具时，你会理解它们为什么这样设计：比如为什么 Cursor 有 Rules 文件、为什么 Claude Code 有 CLAUDE.md
2. 开发 AI 应用时，你会知道一个好的 Agent 不只是"调 API + 写提示词"，还需要考虑工具设计、上下文管理、安全防护等
3. 排查 Agent 行为异常时，你会知道问题可能出在 Harness 的哪个环节：是上下文太长了？是工具描述不清楚？还是安全限制太严了？

8.学习总结

经过这四天的系统学习，我从机器学习与深度学习的理论基础一步步摸索，慢慢理清了大模型从架构诞生、功能迭代到最终工程化落地的完整脉络。整个学习过程就像剥洋葱，从表层的基础模型，到深层的核心技术，最后聚焦到AI Agent落地的关键——Harness。结合自己的学习体会，我把全程的核心内容总结如下，也融入一些个人的理解，方便后续回顾和梳理。

在理论基础阶段，我深入学习了CNN与RNN这些经典模型，说实话，刚开始接触时觉得它们已经很强大了，能解决不少基础的AI任务，但越深入研究越发现，它们的瓶颈其实很明显。CNN擅长处理图像类任务，可面对长文本的语义关联时就显得力不从心；RNN虽然能处理序列数据，却被顺序计算的模式束缚，不仅效率低，长距离依赖的问题也一直难以解决。这些缺陷，在我看来，就是大模型想要进一步发展的“绊脚石”，也让我明白，AI技术的进步，本质上就是不断解决现有问题的过程。

就在我为这些瓶颈感到困惑时，Transformer的出现就像一个盖世英雄，踩着七彩祥云过来说要娶我（娇羞），它的核心创新点直接击中了传统模型的痛点，也让我真正理解了“架构革新”对AI发展的重要性。在我看来，Transformer之所以能成为大模型的基础，关键就在于它解决了之前一直无法突破的核心问题，具体来说有三点：

1. 解决了长距离依赖问题：通过自注意力机制，句子中的每个词都能直接关联其他任意词，无需逐步传递信息，大幅提升了语义理解的完整性；
2. 实现了并行计算：抛弃 RNN 顺序处理的弊端，编码器端可完全并行运算，极大提升了模型训练和推理的效率；
3. 具备良好的可扩展性：架构简洁清晰，可通过增加网络层数、扩大参数规模实现性能提升，为后续大模型的规模化发展提供了可能。

Transformer架构落地后，我能明显感受到大模型的发展进入了“快车道”——参数规模和训练数据量不断突破，模型也慢慢展现出“涌现”现象，这种“突然拥有类人智能”的变化，至今想起来都觉得很神奇。结合自己的学习感悟，我梳理出一条清晰的主线，能直观看到大模型从基础架构到初步成熟的演进过程：

Transformer 提供了架构基础 → 预训练让模型学到海量通用知识 → 指令微调让模型听懂人类需求、精准响应 → RLHF（强化学习从人类反馈）让模型输出更贴合人类偏好、更“好用” → Scaling Law（缩放定律）驱动模型参数与性能同步提升、越做越大 → MoE（混合专家模型）解决大模型规模化的成本与效率难题，让“做大”可控 → 稀疏注意力机制突破长文本处理瓶颈 → 量化与蒸馏技术降低模型部署门槛，让大模型从云端走向个人终端，实现普及。

当大模型具备了基础智能，我开始思考一个问题：明明模型很强大，为什么有时候输出的结果还是不符合预期？后来才明白，核心是我们没有找对和大模型“沟通”的方式。这时候，Prompt Engineering（提示词工程）就应运而生了。在我看来，它的核心价值不是“教模型做事”，而是“教我们如何和模型对话”——通过精准的提示词设计，引导模型规避偏差，精准匹配我们的需求，让大模型的能力真正发挥出来，这也是我在实际使用大模型时最有体会的一点。

但即便掌握了提示词工程，我发现大模型依然有两个无法回避的硬伤：知识过期和幻觉。比如让它回答最新的行业动态，它可能还停留在几年前的信息；有时候它会一本正经地输出错误内容，让人难以分辨。而RAG（检索增强生成）技术的出现，完美解决了这个痛点，它“先查资料再回答”的核心逻辑，在我看来就是给大模型装上了“实时知识库”，让它摆脱了“凭记忆答题”的局限，其完整的作用路径，我结合自己的理解总结为：

大模型有知识过期和幻觉两个硬伤 → RAG 通过“先检索、后生成”的模式精准破局 → Embedding（嵌入）技术将文本转化为机器可理解的语义向量 → 向量数据库实现语义向量的高效存储与快速检索 → 文档切割决定检索粒度，影响检索精准度 → 混合检索+重排序进一步提升检索质量，筛选最优信息 → Prompt 模板约束模型生成行为，确保输出贴合需求 → 高级架构优化让 RAG 系统更智能、更自适应 → 工程实践解决权限管理、成本控制、安全防护等现实落地问题 → 完善的评估体系保障 RAG 系统持续迭代优化 → Text2SQL 技术将 RAG 方法论延伸至结构化数据领域，通过 Schema 检索+Agent 错误重试，实现自然语言直接查询数据库。

当大模型通过RAG解决了核心缺陷，具备了精准响应、实时更新的能力后，我开始思考：AI能不能真正脱离人工干预，自主完成复杂的现实任务？而AI Agent，就是这个问题的答案。但在学习过程中我发现，AI Agent能落地，关键不是大模型本身，而是我们本次学习的核心——Harness。在我看来，Harness就像是AI Agent的“管家”，默默支撑着它稳定运行，也是AI Agent能从“理论”走向“实践”的关键。

结合前文的学习和自己的理解，我终于理清了AI Agent和Harness的关系：AI Agent的核心不是大模型（LLM），而是“大语言模型+外部装备增强”的组合，核心能力就是记忆信息、调用工具、分步执行。而Harness，作为AI Agent的软件基础设施，就相当于给这个“组合体”搭建了一套完整的运行体系，它统管着大模型之外的所有事务，像马具约束马匹、安全带保障安全一样，连接、约束、协同Agent的所有组件。我觉得，Harness最厉害的地方，就是精准解决了大模型“没手脚、会失忆、会跑偏”的痛点，补齐了工具调用、上下文管理、流程控制这三大关键能力，让AI Agent能真正“干活”。

Harness的出现，在我看来并不是偶然，而是AI从简单问答向复杂自主任务升级的必然结果。它的演进路径也很清晰：从ChatGPT插件系统的初步探索，到Anthropic、OpenAI等机构的不断研究积累，最终实现了Harness概念的成熟落地。结合自己的学习，我把它的核心组件总结为5个部分，每个部分都有其不可替代的作用：Agent循环是“闭环引擎”，实现任务的迭代推进；工具系统是“手脚”，让AI能接触真实世界；上下文管理是“记忆”，帮AI记住关键信息；安全系统是“防护盾”，规避危险操作；会话与持久化是“存档器”，保障任务能跨会话持续推进。

其实我一开始觉得，Harness离自己很远，毕竟我不打算从零构建一个Harness，但深入学习后才发现，理解它的核心价值，对我使用和开发AI工具帮助很大。结合自己的实际体验来说：用Cursor、Claude Code等工具时，我不再觉得它们的设计“莫名其妙”，反而能理解Cursor的Rules文件、Claude Code的CLAUDE.md，本质都是Harness组件的具体体现；以后开发AI应用时，我也会明白，一个优秀的Agent不只是“调API+写提示词”，更要考虑工具设计、上下文管理、安全防护这些细节；如果遇到Agent行为异常，我也能快速定位问题——是上下文太长导致失忆？还是工具描述不清楚导致调用出错？或是安全限制太严导致无法执行？这些都是学习Harness带给我的实际收获。

回顾这段时间的学习，我最大的感受就是：AI技术的发展，从来都是“发现问题—解决问题”的循环过程。我们从深度学习基础模型的痛点出发，见证了Transformer如何奠定大模型的架构基础，跟随大模型的演进主线，了解了提示词工程、RAG技术如何一步步完善大模型的能力，最终聚焦于Harness如何支撑AI Agent落地。对我而言，这段学习不仅掌握了相关知识，更让我学会了用“工程化思维”看待AI——AI不只是一个“黑盒子”，它的每一个功能、每一次升级，背后都有一套完整的技术体系支撑。未来，随着Harness技术的持续优化，我相信AI Agent会在更多现实场景中实现自主运行，真正走进我们的工作和生活，这也是我对AI技术未来发展的期待。